BIZEV-99: Best Practice Summary

> **Series:** BIZEV — Business Events & Funnel Analysis | **Notebook:** 99 | **Created:** March 2026 | **Last Updated:** 10/25/2026

Overview

This notebook consolidates every actionable best practice from the BIZEV series (notebooks 01 through 07) into a single, definitive reference. Each practice specifies exactly what to set, with no hedging. Use this as a checklist when instrumenting business events, building conversion funnels, and creating executive reports in Dynatrace.

Table of Contents

1. Data Model and Schema
2. Ingestion and Instrumentation
3. Event Naming and Payload Design
4. Funnel Analysis
5. Revenue and Impact Analysis
6. KPIs and Metric Extraction
7. Executive Reporting and Dashboards
8. DQL Query Patterns
9. SLA and SLO Tracking
10. Gen2 vs Gen3 Adoption

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	`storage:bizevents:read`, `storage:events:read`, `storage:metrics:read`, `bizevents.ingest`
Knowledge	BIZEV-01 through BIZEV-07

1. Data Model and Schema

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Use `bizevents` for business transactions	`fetch bizevents` — never use `events` or `logs` for business analytics	Critical	Data Source
2	Always populate `event.type`	Set to a reverse-domain string (e.g., `com.myapp.order.completed`) on every event	Critical	Schema
3	Always populate `event.provider`	Set to the source application or system name	Critical	Schema
4	Carry the business line in a custom field	Add a payload field such as `product_category` (e.g., `ecommerce`, `payments`); `event.category` is an ITIL significance category	Recommended	Schema
5	Include a `user_id` or `session_id` field	Required for funnel analysis and distinct-user conversion rates	Critical	Schema
6	Include a numeric `amount` field on transaction events	Store as a number, never as a string — enables `sum()`, `avg()` without casting	Critical	Schema
7	Use consistent field types across all events	Always send `amount` as double, `quantity` as long — never vary types	Recommended	Schema
8	Never confuse `bizevents` with `events`	`bizevents` = business transactions; `events` = infrastructure/operational signals	Critical	Data Source

2. Ingestion and Instrumentation

#	Best Practice	Recommended Setting/Value	Priority	Category
9	Start with OneAgent auto-capture for web apps	Configure capture rules in **Settings > Collect and Capture > Business events**	Critical	Ingestion
10	Use CloudEvents format for API ingestion	`Content-Type: application/cloudevents+json` with `specversion`, `type`, `source`, `data` fields	Critical	Ingestion
11	Use batch ingestion for high-volume scenarios	`Content-Type: application/cloudevents-batch+json` — send arrays of events in a single POST	Recommended	Ingestion

#	Best Practice	Recommended Setting/Value	Priority	Category
12	Implement retry logic with exponential backoff	Retry on `429` and `5xx`; keep each request under the documented 5 MB payload limit (no request-rate figure is published)	Recommended	Ingestion
13	Use the RUM and mobile APIs for client-side capture	`dynatrace.sendBizEvent(type, fields)` (RUM JavaScript API) or the mobile agents' equivalent; backend code without a capture rule uses the ingest API — there is no OneAgent SDK method for business events	Recommended	Ingestion
14	Use OpenPipeline for span-to-bizevent mapping	A **Business event** processor in the OpenPipeline **Data extraction** stage emits a new bizevent per matching span — no second instrumentation	Recommended	Ingestion
15	Never double-instrument	If spans already carry business data, extract it with a Business event processor (Data extraction stage) — do not add a second telemetry call in code	Critical	Ingestion
16	Verify ingestion health with a 15-minute interval timeseries	Run `makeTimeseries event_count = count(), interval:15m` daily to detect ingestion gaps	Recommended	Monitoring
17	Validate required field completeness after instrumentation	Query `countIf(isNotNull(event.type)) / total` to confirm >99% field population	Critical	Monitoring

3. Event Naming and Payload Design

#	Best Practice	Recommended Setting/Value	Priority	Category
18	Use reverse-domain notation with action verb	Format: `com.<company>.<domain>.<action>` (e.g., `com.myapp.order.created`)	Critical	Naming
19	Use all-lowercase, dot-separated names	`com.myapp.payment.processed` — never `PaymentProcessed` or `PAYMENT_PROCESSED`	Critical	Naming
20	Never put version numbers in event type names	Use payload fields for versioning — `com.myapp.order.created` stays the same, add `schema_version: 2` in payload	Critical	Naming
21	Never use meaningless identifiers as event types	`event_12345` is forbidden — use descriptive domain names	Critical	Naming

#	Best Practice	Recommended Setting/Value	Priority	Category
22	Include business identifiers in payload	`order_id`, `customer_id`, `session_id`	Critical	Payload
23	Include measurable values in payload	`amount`, `quantity`, `duration_ms`	Critical	Payload
24	Include categorical dimensions in payload	`category`, `region`, `customer_tier`	Recommended	Payload
25	Never include high-cardinality strings in payload	No full URLs, stack traces, or free-text fields — parse into structured fields	Critical	Cardinality
26	Use high-cardinality fields for filtering only, not grouping	`filter order_id == "ORD-123"` is fine; `summarize by:{order_id}` on millions of values is not	Critical	Cardinality
27	Keep `summarize by:` dimensions under 1,000 distinct values	Fields like `customer_tier` (~3 values) or `product_category` (~100 values) are ideal	Recommended	Cardinality

4. Funnel Analysis

#	Best Practice	Recommended Setting/Value	Priority	Category
28	Define explicit funnel steps as distinct event types	Each step gets its own `event.type`: `product.viewed` → `cart.updated` → `checkout.started` → `payment.processed` → `order.completed`	Critical	Funnel Design
29	Use `countIf` for funnel step counts in a single query	`summarize {step1 = countIf(event.type == "..."), step2 = countIf(...)}` — never run separate queries per step	Critical	DQL Pattern
30	Use `countDistinctApprox` for distinct-user funnels	`countDistinctApprox(if(event.type == "...", then: user_id))` — more accurate than event counts for conversion rates	Critical	Funnel Design
31	Calculate step conversion as `step_N / step_N-1 * 100`	`round(toDouble(step2) / toDouble(step1) * 100, decimals: 1)`	Critical	Metrics

#	Best Practice	Recommended Setting/Value	Priority	Category
32	Calculate overall conversion as `last_step / first_step * 100`	`round(toDouble(step5) / toDouble(step1) * 100, decimals: 2)`	Critical	Metrics
33	Identify checkout abandonment with per-user aggregation	`summarize by:{user_id}` then `filter started_checkout > 0 AND completed_payment == 0`	Recommended	Analysis
34	Measure time between funnel steps	Convert timestamps to millis, subtract, divide by 1000 for seconds: `(unixMillisFromTimestamp(step2_time) - unixMillisFromTimestamp(step1_time)) / 1000.0`	Recommended	Analysis
35	Report `avg`, `median`, and `p95` for inter-step timing	All three are needed to understand the distribution of user behavior	Recommended	Analysis
36	Segment funnels by `event.provider` and a business-line field (e.g., `product_category`)	Reveals which channels and business lines convert best	Recommended	Segmentation
37	Track daily conversion rate trends over 7+ days	Use `bin(timestamp, 24h)` with `summarize by:{day}` to measure campaign/release impact (`1d` is rewritten to `24h` with a notification; days are UTC-aligned)	Recommended	Trending

5. Revenue and Impact Analysis

#	Best Practice	Recommended Setting/Value	Priority	Category
38	Correlate detected problems with business event volume	Overlay `fetch dt.davis.problems` timelines with `fetch bizevents` volume to visualize impact	Critical	Incident Impact
39	Use `spread:` for concurrent problem timelines	`makeTimeseries count = count(), spread: timeframe(from: event.start, to: coalesce(event.end, now()))`	Recommended	DQL Pattern
40	Compare incident periods against baselines	Compare same hour yesterday or same day last week using `bin(now(), 24h)` offsets	Critical	Incident Impact

#	Best Practice	Recommended Setting/Value	Priority	Category
41	Filter to business hours for impact assessment	Pass your timezone, or 9–5 is UTC: `getDayOfWeek(timestamp, timezone: "<tz>") <= 5 AND getHour(timestamp, timezone: "<tz>") >= 9 AND getHour(timestamp, timezone: "<tz>") < 17` (ISO days: 1 = Monday)	Recommended	Context
42	Convert Dynatrace Intelligence `resolved_problem_duration` from nanoseconds to hours	`resolved_problem_duration / 1h` — the field is in nanoseconds, not milliseconds	Critical	Data Conversion
43	Exclude duplicate problems from impact analysis	`filter dt.davis.is_duplicate == false` . Do not add `dt.davis.is_frequent_event` — frequent issue detection is being phased out and the flag no longer separates anything (ADOPT-03 §5)	Critical	Data Quality
44	Exclude maintenance windows from SLA calculations	`filter maintenance.is_under_maintenance == false`	Recommended	Data Quality
45	Use `sum(toDouble(amount))` for revenue calculations	Always cast `amount` to double before aggregation	Critical	DQL Pattern
46	Track hourly revenue to detect incident-correlated dips	`makeTimeseries hourly_revenue = sum(toDouble(amount)), interval:1h`	Recommended	Monitoring

6. KPIs and Metric Extraction

#	Best Practice	Recommended Setting/Value	Priority	Category
47	Define KPIs that are Specific, Measurable, Actionable, and Time-bound	Every KPI must answer a concrete business question with a formula	Critical	KPI Design
48	Track these five core KPIs at minimum	Transaction Volume (`count()`), AOV (`avg(amount)`), Conversion Rate, Error Rate, Revenue per Hour (`sum(amount)` per hour)	Critical	KPI Design
49	Distinguish business error rates from HTTP error rates	A payment can fail (business error) with HTTP 200 — use `event.type` to identify business failures, not HTTP status	Critical	KPI Design

#	Best Practice	Recommended Setting/Value	Priority	Category
50	Require minimum sample size before calculating error rates	`filter total > 10` before computing error percentages to avoid misleading rates	Recommended	Data Quality
51	Extract critical KPIs as Dynatrace metrics via OpenPipeline	Counter metric / Value metric processors in the OpenPipeline **Metric extraction** stage for order counts and revenue values	Critical	Metric Extraction
52	Set metric dimensions to low-cardinality fields only	Dimensions: `event.provider`, `product_category` — never `order_id` or `user_id`	Critical	Metric Extraction
53	Use extracted metrics for alerting and SLOs	Extracted metrics enable native Dynatrace Intelligence alerting and SLO definitions — DQL on bizevents does not	Critical	Alerting
54	Build a composite business health score	Weight: Transaction Volume 30%, Error Rate 30%, AOV 20%, Conversion Rate 20% — thresholds: Green/Yellow/Red	Recommended	Health Scoring
55	Health score thresholds: Error Rate	Green: < 1%, Yellow: 1–5%, Red: > 5%	Recommended	Health Scoring
56	Health score thresholds: Volume vs Baseline	Green: > 90%, Yellow: 70–90%, Red: < 70%	Recommended	Health Scoring

7. Executive Reporting and Dashboards

#	Best Practice	Recommended Setting/Value	Priority	Category
57	Lead with business outcomes, not infrastructure metrics	Report transactions, revenue, conversion rates — never CPU or memory to executives	Critical	Presentation
58	Always provide baseline context	Compare against yesterday, last week, or last month — never show a number without comparison	Critical	Presentation
59	Quantify incidents in business terms	"Lost 1,200 transactions and $47,000 revenue" not "Service had 500ms latency for 2 hours"	Critical	Presentation
60	One KPI per dashboard tile	Each tile answers exactly one question — no multi-metric tiles	Recommended	Dashboard Design
61	Include a data freshness indicator	`if(latest_event > now() - 5m, then: "CURRENT", else: "STALE")` on every executive dashboard	Recommended	Dashboard Design
62	Executive dashboard must answer three questions	1) Is the business running? 2) Are there problems? 3) What is the trend?	Critical	Dashboard Design

#	Best Practice	Recommended Setting/Value	Priority	Category
63	Automate report delivery via Workflows	Schedule daily reports at 8 AM weekdays with a Schedule trigger (Fixed time, weekdays — or Cron schedule `0 8 * * 1-5`)	Critical	Automation
64	Use `bin(now(), 24h) - 24h` to `bin(now(), 24h)` for "yesterday" queries	Produces clean 24-hour boundaries for scheduled reports	Recommended	DQL Pattern
65	Include a weekly business review query set	Daily volume trend, problems resolved with MTTR, week-over-week volume comparison, busiest hours	Recommended	Reporting
66	Dashboard tile for current transaction rate	`count() / 15.0` over `from:-15m` = transactions per minute	Recommended	Dashboard Design
67	Limit top-N lists to 10 items max in executive views	`limit 10` on all ranked queries for readability	Optional	Dashboard Design

8. DQL Query Patterns

#	Best Practice	Recommended Setting/Value	Priority	Category
68	Always specify a time range on `fetch bizevents`	`fetch bizevents, from:-24h` — never omit the time range	Critical	Performance
69	Filter early, sort late, limit last	Place `filter` immediately after `fetch`, `sort` after `summarize`, `limit` at the end	Critical	Performance
70	Use `==` for exact matches, `~` only for wildcards	`filter event.type == "com.myapp.order.completed"` not `event.type ~ "*order*"`	Critical	Performance
71	Use `in()` with curly-brace arrays for multi-value filters	`filter in(event.type, {"type1", "type2"})` — never SQL-style `IN ('a','b')`	Critical	Syntax
72	Alias all aggregations used in sort	`summarize c = count()` then `sort c desc` — never `sort count() desc`	Critical	Syntax

#	Best Practice	Recommended Setting/Value	Priority	Category
72b	Wrap multiple `summarize` aggregations in `{}` when using `by:`	With `by:` and 2+ aggregations, wrap them: `summarize { a = count(), b = avg(x) }, by: {field}` . Unwrapped, the tenant emits a `PARAMETERS_SHOULD_BE_GROUPED` INFO (the query still runs). Without `by:` , both forms are canonical and no INFO is emitted.	**Recommended**	Syntax
73	Use `round(..., decimals: N)` with named parameter	`round(value, decimals: 2)` — never `round(value, 2)`	Critical	Syntax
74	Use `if(cond, then: ..., else: ...)` with named parameters	Named `then:` and `else:` parameters are mandatory	Critical	Syntax
75	Cast `amount` fields with `toDouble()` before math	`sum(toDouble(amount))` — payload fields may arrive as strings	Recommended	Data Types
76	Use `isNotNull()` before aggregating optional fields	`filter isNotNull(amount)` before `sum(toDouble(amount))`	Critical	Data Quality
77	Use `countDistinctApprox` over `countDistinct` for large datasets	Faster with negligible accuracy loss for >10,000 distinct values	Recommended	Performance
78	Target specific Grail buckets when possible	`fetch bizevents, bucket: {"my_bizevents_*"}` to reduce scan volume	Optional	Performance

9. SLA and SLO Tracking

#	Best Practice	Recommended Setting/Value	Priority	Category
79	Define a Transaction Success Rate SLO	Target: 99.5% — Formula: `successful / total * 100` from payment events	Critical	SLO
80	Monitor business-event ingestion continuity — do not report it as availability	Count empty hours (no business events) in the periods the business transacts; an empty hour means a quiet period or broken capture, not downtime. Availability comes from the success-rate SLO (#79)	Recommended	SLO

#	Best Practice	Recommended Setting/Value	Priority	Category
81	Define an MTTR SLO	Target: < 2 hours — Formula: `avg(resolved_problem_duration)` in hours, excluding duplicates and maintenance-window problems (#44)	Recommended	SLO
82	Define a Revenue Impact per Incident SLO	Target: < $5,000 — Formula: baseline revenue minus incident-period revenue	Recommended	SLO
83	Do not derive SLA availability from summed problem durations	Problems overlap, so summed durations are entity-hours of impact, not downtime — they can exceed the period many times over (BIZEV-04 §6). Define availability as an SLO on good ÷ total events (SLO-02/03)	Critical	SLA
84	Include SLO status indicator in every report	`if(success_rate >= 99.5, then: "MET", else: "MISSED")`	Critical	Reporting
85	Track MTTR with `avg`, `median`, and `p95`	Report all three — avg shows the mean, median shows the typical case, p95 shows worst cases	Recommended	Metrics

10. Gen2 vs Gen3 Adoption

Practices from BIZEV-07 for environments that have not (fully) moved to the Gen3 experience.

#	Best Practice	Recommended Setting/Value	Priority	Category
86	Confirm the platform generation before designing business analytics	Dynatrace Managed = no business events in any form (classic toolchain only); SaaS with Grail = adopt business events, regardless of how classic the day-to-day operation is	Critical	Adoption
87	On Grail-enabled SaaS tenants, build new business analytics on business events, not classic mechanisms	Calculated service metrics are closed to new customers and superseded by OpenPipeline metric extraction — new classic investment is migration debt	Critical	Adoption

#	Best Practice	Recommended Setting/Value	Priority	Category
88	Use the minimal Gen3 footprint for hybrid adoption	Capture via Settings (OneAgent rules — Full-Stack mode required), report in Notebooks + the new Dashboards app, grant `storage:bizevents:read`; add OpenPipeline only if you need custom buckets or new `bizevents.*` metrics (the classic pipeline is not available to accounts created from September 2026) — nothing else has to move	Recommended	Adoption
89	Bridge business KPIs to classic dashboards with metric extraction	`bizevents.`-prefixed extracted metrics are readable in Data Explorer, classic dashboards, and classic metric-event alerting (classic pipeline: not available to accounts created from September 2026 and to be deactivated in a future release — build new extraction in OpenPipeline)	Recommended	Adoption
90	Create metric-extraction rules at capture time, not reporting time	Extraction is forward-only — no backfill; with classic-pipeline extraction, late-timestamped events land in a `.failed`-suffixed key	Critical	Adoption
91	Capture must-not-miss events server-side, not (only) via RUM	`sendBizEvent` is dropped for unmonitored sessions (cost and traffic controls) — orders and payments belong on OneAgent capture rules or the ingest API	Critical	Adoption
92	On Managed, apply reverse-domain naming to classic artifacts	Name request attributes, session properties, and log metrics in `com.<company>.<domain>.<action>` style so the eventual migration is a mapping exercise	Recommended	Adoption
93	Size business events against your actual license model before rollout	Classic DDU: 100 DDUs/GB ingest, 0.30 DDUs/GB-day retain, 1.70 DDUs/GB query, 200,000 DDUs/year free tier; DPS: Events powered by Grail capability	Recommended	Adoption
94	Scope business-analyst access with a bucket-limited IAM policy	`ALLOW storage:bizevents:read WHERE storage:bucket-name = "default_bizevents";` plus the matching `storage:buckets:read` statement — grants Gen3-app access to business events only, nothing else in Grail; route sensitive streams to custom buckets for per-team policies	Recommended	Adoption

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.